Joule–Thomson cooler model (He-3 and other fluids): theory, code, usage and validation

After Zhang & Dang, *Cryogenics* **116** (2021) 103282

Part 0 Overview

Purpose. The project models precooled Joule–Thomson (JT) coolers made of n counterflow heat exchangers (CHEX) with precoolers (PHEX) between them, an orifice and an evaporator. It reproduces the He-3 cooler of Zhang & Dang, and extends the same energy-balance approach to (i) coolers from other papers, (ii) other fluids (any CoolProp fluid) and (iii) your own designs. Everything is driven from one command-line script, `jt_run.py`.

What it computes. From T_C (cold-end temperature), P_U (high pressure), the gas temperature leaving each precooler $T_{\mathrm{pre},j}$, the effectiveness η_i of each CHEX and either an orifice diameter d or a measured mass flow, it returns: all state points (T, P, h, phase); the specific refrigeration q_C; the mass flow $\dot{m}$, the refrigeration Q_C and every precooler load $Q_{\mathrm{pre},j}$; the optimal P_U and $T_{\mathrm{pre,last}}$ for q_C and Q_C; sensible upstream precooler temperatures when they are unknown; and a verdict on whether the design is physically possible.

Three workflows.

1. *Validate the paper:* reproduce Zhang & Dang's optimum, fits, mass flux and Q_C with the supplied He-3 tables and score them (`validate`).
2. *Another paper's cooler:* write a case file from a paper's inputs, with its reported values and a data reference, and compare (`solve`).
3. *Your own design:* create a named case, check feasibility, solve or optimise, and keep every run linked to its configuration (`new`, `feasible`, `run`, `runs`).

Layers.

Layer	Content
Properties	`jt/properties.py` (He-3 from CSV, interpolation), CoolProp adapter in `jt/cycle_core.py`; both expose the same interface (`h`, `s`, `rho`, `Psat`, `T_from_Ph`, ...)
Cycle	`jt/cycle_core.py`: states, CHEX solver, `solve_cycle`, orifice choked-flow model, flags
Optimisation	`jt/design_optimize.py` (last stage), `jt/phex_load_map.py` (PHEX loads), `jt/workflow.py` (full pipeline, upstream choice, feasibility, run storage)
Front door	`jt_run.py`: `check`, `validate`, `new`, `feasible`, `run`, `runs`, `solve`, `optimize`, `phex`
Validation	`reproduce_zhang_dang_2021.py`, `tests/`, the case files in `cases/`

Symbols. h specific enthalpy; P_U high pressure; $P_D = P_{\mathrm{sat}}(T_C)$ low pressure; T_H warm-end temperature (300 K by default); η_i effectiveness of CHEX i; $T_{\mathrm{pre},j}$ gas temperature leaving PHEX j; q per unit mass flow [J/kg], $Q = \dot{m}\, q$ power [W]; q_C, Q_C refrigeration at the evaporator; $q_{\mathrm{pre},j}$, $Q_{\mathrm{pre},j}$ heat removed at PHEX j.

Part I Theory and where it lives in the code (sections 1–9)

1. The cycle, the notation and the assumptions

High-pressure gas enters at the warm end (T_H, pressure P_U), passes through n counterflow heat exchangers (CHEX) with a precooler (PHEX) between consecutive ones, and is throttled isenthalpically through a small orifice to the evaporator at T_C. The low-pressure stream leaves the evaporator as saturated vapour and flows back through the same CHEXs in the opposite direction. Everything below is per unit mass flow unless stated.

For CHEX i (numbered warm to cold): A_i = hot inlet, B_i = hot outlet, C_i = cold inlet, D_i = cold outlet. The cold stream passes on, $C_{i-1} = D_i$, and the hot stream passes through PHEX $i-1$ between B_{i-1} and A_i. B_n is

the JT-valve inlet and C_n is the evaporator outlet. In the paper's labelling of the last stage, $A_n = 7$, $B_n = 8$, $C_n = 10$, $D_n = 11$.

- Low-side pressure $P_D = P_{\text{sat}}(T_C)$; no pressure drop in any exchanger; discharge coefficient $C_d = 1$; no heat leak.
- $C_n = (P_D,$ saturated vapour at $T_C)$, so $h_{C_n} = h_g(T_C)$. PHEX outlet temperatures $T_{\text{pre},j}$ are *prescribed*: $A_1 = (P_U, T_H)$ and $A_{j+1} = (P_U, T_{\text{pre},j})$.
- The JT valve is isenthalpic: $h_{\text{valve,out}} = h_{B_n}$ at pressure P_D.
- Mass flow is never an input. It is set by the orifice (section 7) or given as a measured value (section 12): $\dot{m} = \frac{\pi}{4} d^2 G$.

Specific and total refrigeration, with the evaporator absorbing the enthalpy rise from valve outlet to saturated vapour:

$$q_C = h_g(T_C) - h_{B_n}, \qquad Q_C = \dot{m}\, q_C. \tag{1}$$

Code: `State`, `state_PT`, `state_Ph`, `saturated_vapour_state`, `state_labels` (state numbering) in jt/cycle_core.py.

2. What limits a counterflow heat exchanger

Let Q be the heat transferred per unit mass flow. With equal flow rates on both sides and no pressure drop, energy conservation on each stream gives

$$h_B = h_A - Q, \qquad h_D = h_C + Q. \tag{2}$$

Two temperature-feasibility conditions bound Q:

- the cold stream cannot leave warmer than the hot inlet: $T_D \le T_A$, i.e. $h_D \le h(P_D, T_A)$, so $Q \le W_{\text{warm}} \equiv h(P_D, T_A) - h_C$;
- the hot stream cannot leave colder than the cold inlet: $T_B \ge T_C$, i.e. $h_B \ge h(P_U, T_C)$, so $Q \le W_{\text{cold}} \equiv h_A - h(P_U, T_C)$.

$$Q \le Q_{\max} = \min(W_{\text{warm}},\, W_{\text{cold}}). \tag{3}$$

The warm-end limit comes from the *low-pressure* isobar and the cold-end limit from the *high-pressure* isobar; they differ because the two streams have different heat capacities at their different pressures. Which one binds depends on the operating point.
Code: `chex_limits` returns $(Q_{\max}, W_{\text{warm}}, W_{\text{cold}},$ binding end), where the binding end is `warm_end`, `cold_end` or `both`.

3. Effectiveness and the exchanger equations

Effectiveness is the ratio of actual to maximum possible heat transfer:

$$\eta = \frac{Q}{Q_{\max}} \implies Q = \eta\, Q_{\max}, \qquad 0 \le \eta \le 1. \tag{4}$$

Combining with (2) gives both outlet enthalpies, and the outlet states follow from the table by inverting $h(P,T)$ at fixed pressure:

$$h_B = h_A - \eta Q_{\max}, \quad h_D = h_C + \eta Q_{\max}, \quad T_B = T(P_U, h_B), \quad T_D = T(P_D, h_D). \tag{5}$$

Because $Q \le Q_{\max}$ by construction, any $\eta \in [0,1]$ yields $T_B \ge T_C$ and $T_D \le T_A$. In particular $\eta = 1$ is always an admissible exchanger.
Relation to the paper. Zhang & Dang define the last-stage effectiveness on the cold side only, $\eta = (h_D - h_C)/(h(P_D, T_A) - h_C)$, i.e. Q/W_{warm}. This equals (4) whenever $W_{\text{warm}} \le W_{\text{cold}}$ (at or below the optimum P_U). Above the optimum $W_{\text{cold}} < W_{\text{warm}}$ and the paper's formula asks for $Q > W_{\text{cold}}$, i.e. $h_B < h(P_U, T_C)$, so $T_8 < T_C$: the hot stream would leave colder than the cold stream's inlet, which is unphysical. Using (4) removes this branch.
Code: `_chex_solve` evaluates $Q = \eta \max(Q_{\max}, 0)$, h_B, h_D, then `state_Ph` for B and D; `chex_AC` is the public wrapper.

4. The last stage: why $q_C = \min(\Delta h_T, q_L)$

For the last CHEX, $A = (P_U, T_A)$ with $T_A = T_{\mathrm{pre},n-1}$ (state 7) and $C = h_g(T_C)$. Using $q_C = h_C - h_B$ and $h_B = h_A - Q$:

$$q_C = (h_C - h_A) + Q \;\le\; (h_C - h_A) + \min(W_{\mathrm{warm}}, W_{\mathrm{cold}}). \tag{6}$$

Substituting the two limits and defining

$$\Delta h_T \equiv h(P_D, T_A) - h(P_U, T_A), \qquad q_L \equiv h_g(T_C) - h(P_U, T_C), \tag{7}$$

we get $(h_C - h_A) + W_{\mathrm{warm}} = \Delta h_T$ and $(h_C - h_A) + W_{\mathrm{cold}} = q_L$. Therefore

$$q_C \le \min(\Delta h_T,\, q_L), \qquad \text{with equality if and only if } \eta = 1. \tag{8}$$

This is the statement "if $A \le B$ and $A \le C$ everywhere, then $A = \min(B, C)$ when it is saturated". The bound is a pure inequality; the effectiveness is what converts it to an equality ($\eta = 1 \Rightarrow Q = Q_{\max}$).

Physical meaning. Δh_T is the isothermal enthalpy difference between the low and high pressure at T_A: how much cooling the real-gas expansion provides when the exchanger is warm-limited. q_L is the enthalpy needed to bring the stream from the cold-end state (P_U, T_C) to saturated vapour: the cooling available when the exchanger is cold-limited.

Trends and the optimum. At fixed T_A and T_C:

- q_L falls with P_U, because $(\partial h/\partial P)_T = v(1 - \beta T)$ is positive for the liquid-like state at T_C (here $\beta T \ll 1$), so $h(P_U, T_C)$ rises until it reaches h_g. That happens at $P_{U,\max}$ (≈ 1.13 MPa at 1 K), where $q_L = 0$.
- Δh_T rises with P_U while T_A lies below the inversion curve, because there $\beta T > 1$ makes $h(P_U, T_A)$ decrease with pressure.

One quantity rises and the other falls, so $\min(\Delta h_T, q_L)$ peaks where they cross:

$$\Delta h_T = q_L \quad \text{(paper Eq. 34)}, \qquad P_U < P_{U,\mathrm{opt}}:\ \text{warm-limited}, \quad P_U > P_{U,\mathrm{opt}}:\ \text{cold-limited}. \tag{9}$$

5. Effect of $\eta < 1$ on the last stage

For $\eta < 1$ the same substitution gives, in both regimes,

$$q_C = \eta\, \min(\Delta h_T, q_L) - (1 - \eta)\,(h_A - h_C). \tag{10}$$

(Warm-limited: $(1-\eta)(h_C - h_A) + \eta\,\Delta h_T$; cold-limited: $(1-\eta)(h_C - h_A) + \eta\, q_L$.) The penalty is $(1 - \eta)$ times the large enthalpy gap $h_A - h_C$ between the 8 K gas and the 1 K saturated vapour (≈ 40 kJ/kg for $P_U = 0.36$ MPa), against $q_C \approx 8$ kJ/kg. So $\partial q_C/\partial\eta \approx q_C + (h_A - h_C) \approx 49$ kJ/kg and a drop of 0.01 in η costs about 6% of q_C. Because $Q < Q_{\max}$, T_{B_n} rises above T_C, which also lowers the orifice flux G; together these give the roughly 21% drop in Q_C at $\eta_4 = 0.97$ reported by the paper (our result: 20.8%).

Code: `evaluate_last_stage` (jt/design_optimize.py) is a thin wrapper over `solve_cycle` for a one-CHEX cycle with T_{pre} fixed; it returns q_C, T_8, G, $\dot m$, Q_C, Δh_T, q_L and the binding wall. `find_optimum_pu` maximises q_C over P_U (coarse grid, then golden-section), `find_pu_max` finds the root of $q_L = 0$, `find_tpre_opt` finds the optimal T_{pre} at fixed P_U (where $\Delta h_T = q_L$), and `stage_limits`, `stage_sweep` give the curves behind Figs. 5 and 7.

6. The full cascade and the global energy balance

C_n is known, so the CHEXs are solved from cold to warm. For $i = n, \ldots, 1$: set $A_i = (P_U, T_{\mathrm{in},i})$ with $T_{\mathrm{in}} = (T_H, T_{\mathrm{pre},1}, \ldots, T_{\mathrm{pre},n-1})$, solve (B_i, D_i) from section 3, and pass $C_{i-1} = D_i$ on. Then

$$h_{B_n} \text{ is the valve inlet}; \qquad h_{\mathrm{valve,out}} = h_{B_n}; \qquad q_C = h_g(T_C) - h_{B_n}; \qquad q_{\mathrm{pre},j} = h_{B_j} - h_{A_{j+1}}. \tag{11}$$

Summing the CHEX balances (2) and telescoping along both streams gives an independent consistency check (the energy balance of the whole machine, heat in q_C at the evaporator, heat out $\sum q_{\mathrm{pre}}$ at the precoolers):

$$q_C = \sum_{j=1}^{n-1} q_{\mathrm{pre},j} + (h_{D_1} - h_{A_1}). \tag{12}$$

The residual of (12) is stored as `energy_residual` and a flag is raised if it exceeds a relative tolerance. With $\dot m$ known, $Q_C = \dot m q_C$ and $Q_{\mathrm{pre},j} = \dot m\, q_{\mathrm{pre},j}$.

Code: `solve_cycle`; `CycleResult` holds the states, q_C, q_{pre}, $\dot m$, flags and the binding end of each CHEX. With $n = 1$ it reduces to the last-stage problem of section 4.

7. Orifice mass flux: homogeneous equilibrium model (HEM)

Treat the orifice as an adiabatic, isentropic nozzle with two phases in equilibrium. With inlet stagnation state $(P_0, T_0) = (P_U, T_{B_n})$, $h_0 = h(P_0, T_0)$, $s_0 = s(P_0, T_0)$, energy conservation and isentropy give, at any pressure P along the expansion,

$$u(P) = \sqrt{2\,[h_0 - h(P, s_0)]}, \qquad G(P) = \rho(P, s_0)\, u(P). \tag{13}$$

If s_0 lies between the saturated liquid and vapour entropies at P the state is two-phase:

$$x = \frac{s_0 - s_l}{s_v - s_l}, \qquad h = h_l + x(h_v - h_l), \qquad \frac{1}{\rho} = \frac{1-x}{\rho_l} + \frac{x}{\rho_v}. \tag{14}$$

Otherwise $h(P, s_0)$ and ρ come from the single-phase isobar at P, solving $s(T) = s_0$ for T. The mass flux is limited (choked) at the maximum of $G(P)$, which is where $\mathrm{d}G/\mathrm{d}P = 0$, equivalent to $u = a$ (local equilibrium speed of sound):

$$G = \max_{P_D \le P \le P_0} G(P), \qquad \dot{m} = \tfrac{\pi}{4} d^2 G. \tag{15}$$

Code: `critical_mass_flux_HEM` samples 500 log-spaced pressures between P_D and P_0, refines the interior maximum by golden-section search, reports the throat pressure and whether the flow is choked (maximum strictly inside the range; a maximum at P_D raises `UNCHOKED_RESTRICTION_FLOW`), and checks $u \approx a$. `mass_flow` multiplies by the area. The backend must supply `h_from_Ps` and `rho_from_Ps`.

Comparison with the paper. Zhang & Dang use a real-gas sonic flux with a correction factor Γ and fit $G = \alpha P_U^{\beta} + \gamma$ (Eq. 49, coefficients in Table 2 for $T_C = 1, 2, 2.6, 3$ K), evaluated with the orifice inlet at $T_0 = T_C$. We ignore Γ and compare our HEM $G(P_U)$ at $T_0 = T_C$ with that fit. `paper_mass_flux_at_temperature` evaluates Eq. 49, interpolating between the tabulated T_C curves; `mass_flux_vs_pu` generates our curve.

8. Reproducing the paper's fits (how "our method is solid" is tested)

For each $T_{\mathrm{pre},3}$ we find our $P_{U,\mathrm{opt}}$ and $q_{C,\mathrm{opt}}$ (sections 4–5), then refit the paper's functional forms and compare coefficients and residuals:

$$P_{U,\mathrm{opt}}(T_{\mathrm{pre},3}) = \textstyle\sum_{k=0}^{4} a_k\, T_{\mathrm{pre},3}^{\,k}, \quad a = (-0.102,\ 0.074,\ -2.37\times10^{-3},\ 4.88\times10^{-5},\ -4.30\times10^{-7}) \text{ [MPa] (Eq. 36)}, \tag{16}$$

$$q_{C,\mathrm{opt}} = 11.9 - 11.5\, P_{U,\mathrm{opt}} + 0.027\, T_{\mathrm{pre},3} \ \text{ [kJ/kg] (Eq. 35)}, \tag{17}$$

$$G = \alpha P_U^{\beta} + \gamma \ \text{ (Eq. 49)}. \tag{18}$$

(At $T_{\mathrm{pre},3} = 8$ K these give 0.3615 MPa and 7.96 kJ/kg, the paper's optimum (0.36 MPa, 8.0 kJ/kg).) Differences are expected to be small and come mainly from table interpolation and from HEM against the sonic-flux model.

Code (reproduce_zhang_dang_2021.py): `fit_polynomial` (least-squares polynomial), `fit_plane` (linear fit in two variables), `fit_power_law` (three-parameter $\alpha P^{\beta} + \gamma$ by nonlinear least squares), `error_metrics` (mean and worst relative error), `section_fits`, `section_fig5` (Δh_T, q_L and q_C against P_U at $T_C = 1$ K, $T_{\mathrm{pre},3} = 8$ K), `section_fig7` (Δh_T and q_C against T_{pre} at several P_U), `section_flux`, `section_fig15`. `TestFits` are the unit tests for the fit helpers (`--self-test`).

9. Internal pinch check (flag only, no c_p needed)

Matching end temperatures ($T_B \ge T_C$, $T_D \le T_A$) does not guarantee that the hot stream stays warmer than the cold stream *inside* the exchanger, because c_p differs on the two isobars and varies strongly with T. Take the cross-section where the cold stream has already absorbed heat q, counted from the cold end. Energy conservation gives the enthalpy of each stream there, for $q \in [0, Q]$:

$$h_{\mathrm{cold}}(q) = h_C + q \ \text{ (at } P_D), \qquad h_{\mathrm{hot}}(q) = h_B + q \ \text{ (at } P_U), \qquad \Delta T(q) = T(P_U, h_{\mathrm{hot}}) - T(P_D, h_{\mathrm{cold}}). \tag{19}$$

At $q = 0$ this is $T_B - T_C$ and at $q = Q$ it is $T_A - T_D$. We evaluate ΔT at 20 equal steps in q, using only the table's $h \to T$ inversion (no derivative, no c_p). If $\Delta T < -10^{-3}$ K anywhere, we raise `INTERNAL_TEMPERATURE_CROSSOVER` with the location. It is deliberately coarse and never changes a result.

Code: `chex_profile_check` (interior) and `chex_terminal_flags` (end temperatures), both called from `solve_cycle`.

Part II Design workflow: optimum, upstream stages, feasibility (sections 10–14)

10. Optimum of the last stage: P_U, $T_{\mathrm{pre,last}}$, q_C and Q_C

Section 4 gives $q_C = \min(\Delta h_T, q_L)$ for $\eta = 1$. Two curves matter in practice.

At a given P_U, what is the best $T_{\mathrm{pre,last}}$? Δh_T *falls* as T_A rises (the gas gets warmer and closer to ideal-gas behaviour), q_L does not depend on T_A. Hence as T_A increases q_C stays on the plateau $q_C = q_L$ as long as $\Delta h_T \geq q_L$ and then drops with Δh_T. The optimal precooling temperature is the *warmest* T_A on the plateau,

$$T_{\mathrm{pre,opt}}(P_U): \quad \Delta h_T(P_U, T_{\mathrm{pre,opt}}) = q_L(P_U), \tag{20}$$

because every colder T_A gives the same q_C at a higher precooling cost. (Zhang & Dang's Fig. 7.) For the paper's cooler at $P_U = 0.36$ MPa, $T_{\mathrm{pre,opt}} \approx 8$ K, and Q_C falls from 18.6 mW at 8 K to 13.1 mW at 10 K. q_L falls with P_U and vanishes at $P_{U,\max}$, where $h(P_U, T_C) = h_g$.

At a given $T_{\mathrm{pre,last}}$, what is the best P_U? Eq. (9): the crossing $\Delta h_T = q_L$.

With both free. q_C alone has no interior optimum: it keeps increasing toward low P_U and low $T_{\mathrm{pre,last}}$ (q_L grows as P_U falls, while the mass flow vanishes). The meaningful objective is $Q_C = \dot{m}\, q_C$, with $\dot{m}$ from the orifice (section 7), which rises strongly with P_U. The pipeline therefore reports both optima: the maximum of q_C over the plane $(P_U, T_{\mathrm{pre,last}})$ and the maximum of Q_C. Because Q_C is almost flat along the ridge $\Delta h_T = q_L$, the code also reports a *practical* optimum, the warmest $T_{\mathrm{pre,last}}$ whose Q_C is within 0.5 % of the maximum, as the cheaper precooling choice. The user chooses which one drives the rest of the pipeline (`drive`: `Q_C`, `Q_C_practical` or `q_C`).

Code: `find_optimum_pu`, `find_pu_max`, `find_tpre_opt`, `stage_sweep`, `optimize_last_stage` in jt/design_optimize.py (coarse 2-D grid, default 31×31 for the q_C map and 15×15 for the Q_C map, then refinement); `run_optimize` in jt/workflow.py. If a result sits on a search bound (for example the maximum Q_C at the highest allowed P_U) a note says so and the range should be widened.

11. Upstream precooler temperatures (criterion b)

Invariance. The last CHEX sees only its two boundary states $A_n = (P_U, T_{\mathrm{pre},n-1})$ and $C_n = h_g(T_C)$, so q_C, $\dot{m}$ and Q_C depend on the *last* precooling temperature only. The warmer $T_{\mathrm{pre},j}$ ($j < n-1$) cannot change them; they only redistribute the heat removed among the precoolers. For $\eta = 1$ with all CHEXs warm-limited the total $\sum_j Q_{\mathrm{pre},j}$ is also fixed by the global balance (12) (26.8 mW for the paper's cooler); if a CHEX becomes cold-limited the total rises slightly.

Choice. A user who optimises P_U and $T_{\mathrm{pre,last}}$ typically knows nothing about the warmer levels. Criterion (b), chosen in the design discussion, is to minimise the ideal work needed to lift the precooling loads to T_H (Carnot-weighted load),

$$W = \sum_{j=1}^{n-1} q_{\mathrm{pre},j}\Big(\frac{T_H}{T_{\mathrm{pre},j}} - 1\Big), \qquad T_H > T_{\mathrm{pre},1} > \cdots > T_{\mathrm{pre},n-1}\ \text{(fixed)}, \quad q_{\mathrm{pre},j} \geq 0. \tag{21}$$

(Here $T_{\mathrm{pre},j}$ is the temperature at which PHEX j rejects its load.) The unknown levels are found by differential evolution within the ordering and non-negative-load constraints. For the paper's cooler with $T_{\mathrm{pre,last}} = 8$ K it places the upstream levels near 25.6 K and 12.3 K, with loads of about 9.9, 8.3 and 8.6 mW.

Sensitivity map. For each free upstream level the code also scans that one temperature (41 points) with the others fixed, and plots $Q_{\mathrm{pre},j}$ of every precooler and the Carnot cost against it (`upstream_map.png`, `upstream_map.csv`), so the chosen level can be judged and changed.

Code: `optimise_upstream`, `_carnot_cost`, `upstream_map`, `_plot_map` in jt/workflow.py.

12. Measured mass flow, and effectiveness from measured temperatures

Other papers usually quote the measured mass flow, and their restriction (multi-orifice element, capillary, needle valve) cannot be described by one orifice diameter. A case may therefore give `mdot_mg_s` *instead of* `d_um` (giving both is an error). Then $\dot{m}$ is prescribed, no throat data are produced, and $Q_C = \dot{m}\, q_C$, $Q_{\mathrm{pre},j} = \dot{m}\, q_{\mathrm{pre},j}$ as before. `solve_cycle(\dots,mdot_given=...)`.

If a paper quotes CHEX end temperatures rather than η, the effectiveness follows from (4). For a warm-limited exchanger (return gas leaves at $T_D < T_A$)

$$\eta = \frac{h(P_D, T_D) - h_C}{h(P_D, T_A) - h_C}, \tag{22}$$

and for a cold-limited one (hot gas leaves at $T_B > T_C$) $\eta = (h_A - h(P_U, T_B))/(h_A - h(P_U, T_C))$. Example: Liu et al. quote a supply entering CFHX3 at 17.3 K and the return gas leaving at 17.1 K; at $P_D = 0.104$ MPa this gives $\eta = 0.9857$, which is what makes the model reproduce their $Q_{\max}$ (Part V). Taking $\eta = 1$ there overpredicts q_C by 19 %.

13. The feasibility check

`check_config` (jt/workflow.py) decides whether a configuration can run and whether it makes physical sense. It uses cheap property lookups followed by one trial solve, never raises, and returns a verdict (*feasible*, *feasible with warnings*, *infeasible*) with the reasons. In order:

1. **Fluid window.** T_C must lie inside the supported range: for He-3 the table range 0.5–3.2 K; for CoolProp fluids from the triple point to $0.995\,T_{\text{crit}}$. Then $P_D = P_{\text{sat}}(T_C)$ is read.
2. **Ordering.** Given T_{pre} must decrease strictly from T_H and stay above T_C (blanks are skipped).
3. $P_U > P_D$**.**
4. **Can the stream reach saturated vapour?** $q_L = h_g(T_C) - h(P_U, T_C) > 0$. If $q_L \le 0$ no refrigeration is possible at any T_{pre} (section 4); the message gives $P_{U,\max}$.
5. **Does throttling cool at** $T_{\text{pre,last}}$**?** $\Delta h_T = h(P_D, T) - h(P_U, T) > 0$ (otherwise T is above the inversion temperature). It also finds $T_{\text{pre,opt}}$ (section 10) and warns when the given $T_{\text{pre,last}}$ is warmer, because q_C is then limited by Δh_T and falls.
6. **Trial solve** once all inputs are known. Flags are classified: *errors* are a precooler that would have to heat the gas (negative PHEX load), no net refrigeration, a failed global energy balance; *warnings* are internal pinch crossover, a choking check and unchoked flow.

It checks physical admissibility inside the model (no pressure drop, no heat leak, $C_d = 1$). It does not check hardware (whether a precooler stage can deliver the load) and does not say a design is *good*.

14. Fluids

He-3 uses the CSV tables. Any other fluid goes through CoolProp: He-4 ("Helium", labelled `Helium_4` in file names), Neon, Hydrogen, Parahydrogen, Nitrogen, Argon, Oxygen, Methane. Starting values and optimiser ranges are in `DEFAULTS` (jt/workflow.py) and are editable per run. **Limitation:** for these fluids P_U must exceed the critical pressure (Ne 2.66, H_2 1.30, N_2 3.40 MPa; He-4 0.228 MPa). Below it the hot stream passes through saturation at P_U inside a CHEX and CoolProp raises a saturation error (seen for Neon at $P_U = 2$ MPa). This is a model limitation, not a design fault.

Part III Code structure, data handling and caveats

Folder structure

```
project/
  README.md                    introduction and how to use
  jt_run.py                    command line front door
  reproduce_zhang_dang_2021.py fits, figures, flux, self-tests (used by validate)
  requirements.txt
  jt/
    cycle_core.py              State, CHEX solver, solve_cycle, HEM flow, CoolProp adapter
    properties.py              Helium3Fluid: CSV-backed interpolation
    design_optimize.py         last-stage optimiser, Eq. 49, file-name scheme
    phex_load_map.py           PHEX load map vs warmer precooler temperatures
    workflow.py                configs, naming, feasibility, optimiser pipeline, run storage
  data/he3/                    SinglePhase_He_3.csv, Saturation_He_3.csv, README.md
  cases/                       _template.json, Zhang & Dang (2), Ma 2019 (4), Liu 2017 (1),
                               and every case you create
```

tests/	test_cycle_core, test_interpreter_with_coolprop, test_jt_run, test_workflow (29 tests)
results/	generated: validation/, cases/, runs/<config>/<stamp>_<kind>/

Module map

Module	Main contents
cycle_core	State, state_PT, state_Ph, saturated_vapour_state, state_labels; chex_limits, _chex_solve, chex_AC; solve_cycle (with d or mdot_given); critical_mass_flux_HEM, mass_flow; chex_profile_check, chex_terminal_flags; CoolPropFluid, make_fluid; CycleResult
properties	Helium3Fluid: h, s, rho, Psat, T_from_Ph, h_from_Ps, rho_from_Ps; table loader with header normalisation; phase classification; OutOfRange, DataError
design_optimize	evaluate_last_stage, stage_limits, stage_sweep, find_optimum_pu, find_pu_max, find_tpre_opt, optimize_last_stage, paper_mass_flux_at_temperature, mass_flux_vs_pu, _build_run_stem
phex_load_map	PHEX load map at a fixed JT operating point, sweeping the warmer precooler temperatures
workflow	norm_cfg, config_id, config_hash, save_config; get_fluid, fluid_limits, default_ranges; check_config; run_solve, run_optimize, optimise_upstream, upstream_map; new_run, finish_run, list_runs; plotting helpers
jt_run	cmd_check, cmd_validate, cmd_new, cmd_feasible, cmd_run, cmd_runs, cmd_solve; scorecard (build_scorecard, TOL); case loader (load_case) and sweep

What happens in `run`

1. The case file is read and normalised (`norm_cfg`): types, ranges, one η per CHEX, $N-1$ temperatures (blank allowed), d xor $\dot{m}$, data reference.
2. If every input is given the run is a *solve*: `check_config`, then `solve_cycle`, then states, loads and the P–h plot. If P_U or any T_{pre} is blank (or `--optimize` is given) it is an *optimisation*: section 10 for P_U and $T_{\mathrm{pre,last}}$, section 11 for the upstream levels, then a solve at the chosen values.
3. Everything is written to `results/runs/<config>/<stamp>_<kind>/` with a copy of the configuration (`config.json`), `meta.json` (configuration hash, kind, headline, data reference), `result.json`, `states.csv`, `ph_cycle.png`, and for optimisation `map_qC.png`, `map_QC.png`, `stage_curves.png`, `upstream_map.png`, `upstream_map.csv`.

Naming and run linking

A configuration is named {Fluid}_{T_C}K_{d}um_{N}_stage, followed by _eta{x} when the last CHEX is not ideal. N is the number of CHEXs (one more than the number of precoolers), d is the orifice diameter in μm (nod if none, or {mdot}mgs for a measured flow), He-4 is written Helium_4 and He-3 Helium_3. Example: Helium_3_1K_20um_4_stage_eta0.97. If a file with that name exists but different content, _v2, _v3, ... is appended. The configuration hash is taken over fluid, T_C, P_U, T_H, T_{pre}, η, d, $\dot{m}$ and the optimiser settings, *not* over the data reference, so adding or correcting a reference later never breaks the link between a run and its configuration. `runs` lists each run with the saved configuration file(s) whose hash matches ("NOT SAVED" if none).

Data reference

Each case may carry `reference`: `citation`, `doi`, `location` (table, figure or page), `kind` (reproduction of a paper, validation against a measured cooler, own design, ...), `note` (assumptions). It records where the *inputs of the case* came from, is stored with every run and shown by `runs`. It has nothing to do with the origin of the property tables.

Property tables: interpolation logic (`jt/properties.py`)

- **Input:** a rectangular grid of h, s, ρ over T and P (0.5–300 K, 1 Pa–3 MPa in the supplied tables) and a saturation table (0.5–3.2 K). Headers are matched after normalisation, so `s [J/kg/K]`, `h liq [J/kg]` and `P [Pa]` style columns are all accepted.

- **Two-step PCHIP** (monotone, no overshoot): on each isotherm, $h, s, \ln\rho$ are interpolated in $\ln P$; the values at the query pressure then form an isobar profile interpolated in T. Inverses (T from h or s) use `brentq` on that profile, so $h \leftrightarrow T$ round-trips.
- **Phase sides:** each isotherm below T_{crit} is split at P_{sat} into a liquid-side and a vapour-side class with saturation anchor nodes (only where the saturation table has data, $T \leq 3.2$ K). Above P_{crit} one supercritical class is used.
- **Critical closure:** the saturation table stops at 3.2 K, below $T_{\mathrm{crit}} = 3.3246$ K. The critical point (3.3246 K, 116.4 kPa) is appended to the T–P_{sat} curve *for classification only*. For P between the highest tabulated P_{sat} (101.7 kPa) and P_{crit} the side whose profile contains the target is chosen; the thin near-critical gap raises `OutOfRange`.
- **No extrapolation.** Requests outside the table raise `OutOfRange`; targets within 10^{-9} relative of a table edge are accepted (for example the $\eta = 1$ outlet at exactly 300 K).
- **Limits:** the liquid profile needs ≥ 3 nodes, so liquid below about 45 Pa (below $T \approx 0.55$ K) is unavailable; $T_C \leq 3.2$ K; accuracy is lowest near the critical point. Hold-out tests at twice the real grid spacing gave a median $|\Delta h| < 2$ J/kg in the cycle range, a 95th percentile of tens of J/kg and a worst case of a few hundred J/kg near the critical point and at 5–13 K.
- Hard-coded He-3 constants: T_{crit}, P_{crit}, molar mass, maximum cold-end temperature.

CoolProp stand-in

`CoolPropFluid` implements the same interface as `Helium3Fluid`, so `make_fluid("Helium")` (He-4) and the other fluids run through every solver function unchanged. `tests/test_interpreter_with_coolprop.py` fills the two CSV layouts from CoolProp He-4 and loads them through `Helium3Fluid`, which tests the interpolation against an independent reference. He-4 numbers are not comparable with the He-3 paper.

Flags

Flags never abort a result: `INTERNAL_TEMPERATURE_CROSSOVER`, `UNCHOKED_RESTRICTION_FLOW` (HEM maximum at the back pressure, typical for liquid-like inlets), `CHOKING_CHECK`, `NEGATIVE_PHEX_LOAD`, `NONPOSITIVE_REFRIGERATION`, `GLOBAL_ENERGY_BALANCE`.

Known limitations (read before trusting a number)

1. **Idealised cycle.** No heat leak, no pressure drop, $C_d = 1$, low-side pressure $= P_{\mathrm{sat}}(T_C)$. Real coolers deliver less; Part V shows by how much in the tested cases.
2. **Internal pinch.** Matching the end temperatures does not guarantee a positive temperature difference inside a CHEX when c_p varies strongly. CHEX 1 of the paper's case often raises `INTERNAL_TEMPERATURE_CROSSOVER` with small gaps (a few hundredths of a kelvin). It is a flag only; its effect on the results was not isolated, but the paper reproduction agrees to within about 1 %.
3. **Flow model.** The orifice uses a homogeneous equilibrium model (HEM), within about 1 % of the paper's Eq. 49 for $T_C = 1$–3 K, but real capillaries, needle valves and multi-orifice elements differ; use a measured $\dot{m}$ for such coolers.
4. **Non-helium fluids at low P_U**, see section 14.
5. **Tables.** He-3 limited to 0.5–3.2 K and to the table accuracy; the near-critical gap is unavailable.
6. **Scorecard tolerances** (`TOL` in jt_run.py) are provisional and looser than the agreement obtained (for example 15 % on the flux where about 1 % is achieved); a regression could pass unnoticed.
7. **Upstream choice** follows criterion (b) only; it is not a global system optimisation with real precooler capacities.
8. **Run time.** A fully blank optimisation takes about 1.5 minutes (the HEM Q_C scan); with $T_{\mathrm{pre,last}}$ given about 25 s.
9. The paper does not give $T_{\mathrm{pre},1}, T_{\mathrm{pre},2}$ (placeholders 80 K, 25 K in the case files); they change precooler loads but not q_C, $\dot{m}$ or Q_C.

Part IV How to use

Install with `pip install numpy scipy pandas matplotlib CoolProp` and run everything from the project folder. The He-3 tables go in `data/he3/`. All commands are `python jt_run.py <command>`.

Command	What it does
`check`	He-3 tables present, readable, consistent (ranges, saturation, $h \leftrightarrow T$ round trip)
`validate [--quick]`	data check, unit tests, paper reproduction, scorecard (`results/validation/`)
`new ...`	create and save a named case file with a data reference; prints its feasibility
`feasible case.json`	verdict and reasons (section 13)
`run case.json [options]`	solve, or optimise when something is blank or `--optimize`; saves the run
`runs`	list saved runs, their configuration and data reference
`solve case.json ...`	plain solve with `sweep` and `reported` comparison
`optimize ...`, `phex ...`	original last-stage optimiser and PHEX load map (`--help`)

Workflow 1: validate the paper

```
python jt_run.py check
python jt_run.py validate --quick      # coarse grids; omit --quick for the full run
```

It stops at the first failing step: data check, unit tests, then the reproduction (Eq. 35/36 fits, Fig. 5, Fig. 7, mass flux, Figs. 15/16) and the scorecard against the paper's numbers. Plots, CSVs, `reproduction_report.json` and `scorecard.json` are written to `results/validation/`.

Workflow 2: another paper's cooler

Write a case file from the paper (copy `cases/_template.json`): inputs from the paper, its reported values under `reported` and a `reference` block, then `python jt_run.py solve cases/<name>.json`. If the paper gives a measured flow use `mdot_mg_s`; if it gives CHEX temperatures instead of effectiveness, convert them (section 12). Output: `results/cases/<name>/` with `states.csv`, `result.json` (inputs, metrics, flags, comparison with the reported values) and `sweep.csv` if a `sweep` block is present. Five such cases are included (Part V).

Workflow 3: your own design

```
# 1. create + name + check feasibility  (use - for anything to be optimised)
python jt_run.py new --fluid he3 --T_C 1.0 --eta 1,1,1,1 --T_pre -,-,- \
       --P_U - --d_um 20 --kind "Own design" --citation "my study"
# 2. is it possible?
python jt_run.py feasible cases/Helium_3_1K_20um_4_stage.json
# 3. optimise (blank inputs), save the filled-in configuration
python jt_run.py run cases/Helium_3_1K_20um_4_stage.json --save-final
# 4. solve a fully specified case; look at the saved runs
python jt_run.py run cases/Helium_3_1K_20um_4_stage_v2.json
python jt_run.py runs
```

Options of `run`: `--drive {Q_C,Q_C_practical,q_C}`, `--flow-model {hem,paper_fit}` (`paper_fit` only for He-3), `--pu-min`, `--pu-max` (MPa), `--tpre-min`, `--tpre-max` (K), `--hem-grid`, `--save-final`. Default search ranges per fluid are in `DEFAULTS`; the optimiser reports when a result sits on a bound, in which case widen the range.

Case file keys

`name`, `fluid` (`he3` or a CoolProp name), `T_C` [K], `P_U` [MPa], `T_H` [K, default 300], `T_pre` [K, $N-1$ values, `null` = optimise], `eta` [N values, warm to cold], `d_um` or `mdot_mg_s`, `reference`, `reported`, `sweep`, `optimize`, `source`, `notes`. Keys starting with an underscore are ignored. `reported` names: `q_C_kJ_kg`, `mdot_mg_s`, `Q_C_mW`, `Q_pre1_mW`, ..., `T_JT_inlet_K`, `P_D_kPa`. `sweep` may scan `T_C`, `P_U`, `T_H`, `d_um`, `eta_last`, `T_pre_last`.

Reading a result

`run` prints, for a solve: q_C, the gas temperature before the valve, P_D, $\dot{m}$, Q_C, the limiting end of each CHEX (`warm_end`/`cold_end`), flags, the state table (T, P, h, phase, role) and the precooler loads $q_{\mathrm{pre},j}$, $Q_{\mathrm{pre},j}$. Example (paper's cooler, $P_U = 0.36$ MPa, $T_{\mathrm{pre}} = (80, 25, 8)$ K, $\eta = 1$, $d = 20\,\mu$m): $q_C = 7.961$ kJ/kg, $\dot{m} = 2.339$ mg/s,

Q_C = 18.62 mW, loads 0.51, 9.70 and 16.61 mW at the 80, 25 and 8 K levels (the 80 and 25 K levels are placeholders). For an optimisation it also prints the optima for q_C, Q_C and the practical Q_C, $T_{\mathrm{pre,opt}}$, notes (bounds, plateau), and the chosen upstream levels.

Worked examples

All with $T_H = 300$ K, $\eta = 1$ unless stated; results from the code.

	Cooler	q_C **[kJ/kg]**	$\dot{m}$ **[mg/s]**	Q_C **[mW]**	**Verdict**
A	He-3, 1.5 K, N=3, $T_{\mathrm{pre}} = (60, 10)$ K, $P_U = 0.5$ MPa, $d = 15\,\mu$m	7.89	1.50	11.82	warnings
A2	as A with $T_{\mathrm{pre,last}} = 9.39$ K ($= T_{\mathrm{pre,opt}}$)	8.63	–	13.13	–
B	He-3, 2.0 K, N=4, $T_{\mathrm{pre}} = (80, 25, 10)$, $P_U = 0.4$, $\eta_4 = 0.99$, $d = 20\,\mu$m	5.52	1.93	10.65	warnings
C	He-4, 4.2 K, N=3, (80, 15) K, $P_U = 0.8$ MPa	6.21	2.90	18.01	warnings
D	Nitrogen, 80 K, N=2, (150) K, $P_U = 15$ MPa	117.8	41.6	4901	feasible
E2	Neon, 27 K, N=3, (150, 60) K, $P_U = 4$ MPa	17.94	–	180.7	warnings
F	Hydrogen, 20 K, N=3, (100, 40) K, $P_U = 2$ MPa	184.6	2.39	440.7	warnings

The warnings in A–C and F say that $T_{\mathrm{pre,last}}$ is warmer than $T_{\mathrm{pre,opt}}$ (A2 shows the gain from moving to it). Designs the check refuses: He-3 with $P_U = 1.2$ MPa ("$q_L \leq 0$", $P_{U,\max} \approx 1.13$ MPa); $T_{\mathrm{pre}} = (80, 90, 8)$ (must decrease from T_H); $T_C = 3.4$ K (above the 3.2 K supported limit); $T_{\mathrm{pre,last}} = 60$ K (above inversion, $\Delta h_T \leq 0$); nitrogen with $P_U = 0.1$ MPa (below $P_D = 0.137$ MPa); Neon with $P_U = 2$ MPa (below its critical pressure).

Tests

`python -m unittest discover -s tests -t .` runs 29 tests: cycle core and CHEX logic, the interpolation against CoolProp, the scorecard, and the workflow layer (naming, saving and versioning, blank temperatures, feasibility messages, run linking, measured flow).

Troubleshooting

`cannot import name 'workflow' from 'jt'`: `workflow.py` must be inside `jt/`. Missing modules: run the `pip install` line. `OutOfRange` for He-3: the request is outside the tables. "Saturation pressure … within 10^{-4} %" for a non-helium fluid: raise P_U above the critical pressure.

Part V Validation

Zhang & Dang 2021 (He-3, $T_C = 1$ K) with the supplied tables

`validate` gives 27 pass, 0 warn, 0 fail.

Quantity	**Ours**	**Paper**
$P_{U,\mathrm{opt}}$ at $T_{\mathrm{pre,last}} = 8$ K [MPa]	0.3584	0.36
$q_{C,\mathrm{opt}}$ [kJ/kg]	7.98	8.0
$P_{U,\max}$ ($q_L = 0$) at 1 K [MPa]	1.131	1.13
$\dot{m}$ at $\eta = 1$ [mg/s]	2.339	2.34
Q_C at $\eta = 1$ [mW]	18.62	18.7
Q_C at $\eta_4 = 0.97$ [mW]	14.74	14.7
Drop in Q_C at $\eta_4 = 0.97$	20.8 %	about 21 %
HEM mass flux vs paper Eq. 49 (T_C = 1–3 K)	within about 1 %	–

Besides these anchors the scorecard checks implementation consistency ($q_C = \min(\Delta h_T, q_L)$ to 10^{-6}, $\Delta h_T = q_L$ at the optimum, no optimum on a search bound) and the fits of Eqs. 35, 36 and 49.

Other coolers (case files in `cases/`)

Five cases from four papers with the same architecture, all He-4 (CoolProp) with a prescribed measured mass flow. The model has no heat leak and no return-line pressure drop, so it is expected to over-predict a measured Q_C; the useful tests are the paper's own idealised numbers (design values, calculated $Q_{\max}$) and the PHEX loads.

Case	Ours Q_C [mW]	Paper [mW]	Difference
Liu 2017, He-4 open cycle ($T_C = 4.25$ K, 1.137/0.104 MPa, 17.3 K, 9.1 mg/s), vs calculated $Q_{\max}$	53.1	52.5 (measured 48.9–49.4)	+1.1 %
Ma 2019 Cryogenics, design (4.5 K, 2.0 MPa, 100/15 K, 10.5 mg/s, $\eta = 0.97$)	90.2	100 (design)	−9.8 %
same with $\eta = 1$	108.8	100	+8.8 %
Ma 2019 Cryogenics, open loop 4×20 μm (4.55 K, 11.08 mg/s)	92.4	100.3	−7.9 %
Ma 2019 Cryogenics, closed loop (5.02 K, 11.02 mg/s)	47.2	102.3	−54 %
Ma 2019 Sci China, 2.65 K (2.71 K, 0.6 MPa, 64.5/14.6 K, $\eta = 0.972/0.973/0.972$, 1.2 mg/s)	4.50	1.48 (design 4)	+204 %

Reading the results.

- **Liu 2017** is a direct test of the central result $Q_{\max} = \dot{m}\,\Delta h_T$. Using the effectiveness derived from the paper's temperatures (17.3 K in, 17.1 K out, $\eta_3 = 0.9857$, section 12) the model gives $q_C = 5.835$ kJ/kg against 5.77 kJ/kg in the paper (the remaining 1 % is property-source difference, CoolProp against their enthalpies). With $\eta = 1$ the result would be 19 % high. The upstream levels and η_1, η_2 are not given and do not affect q_C.
- **Ma 2019 Cryogenics, design.** The paper's 100 mW at 10.5 mg/s needs $q_C = 9.52$ kJ/kg; the model gives 8.59 (at $\eta = 0.97$, the paper's lower bound) and 10.37 (at $\eta = 1$), so the design implies $\eta \approx 0.985$. The precooler loads come out at 207 and 278 mW against the design 350 and 260 mW; the 350 mW first-stage figure evidently includes loads beyond the gas precooling.
- **Ma 2019 Cryogenics, closed loop.** Very sensitive: at 5.02 K the saturation pressure is 0.199 MPa, near the He-4 critical pressure, so q_L is small, whereas the quoted suction pressure was 0.16 MPa. Setting $T_C = T_{\text{sat}}(0.16\text{ MPa}) = 4.745$ K gives 78 mW; with $\eta = 1$ at 5.02 K, 66 mW. The model has no return-line pressure drop, which this case needs. The precooler temperatures are the design values (the paper does not give measured ones), so the case is only indicative.
- **Ma 2019 Sci China.** The model gives 4.5 mW against 1.48 mW measured and 4 mW designed. The paper attributes the shortfall to lower mass flow, higher suction pressure and heat leakage, none of which the model contains. The quoted 12 kPa suction pressure (about 13.3 kPa at the evaporator) compares with our $P_{\text{sat}}(2.71\text{ K}) = 15.1$ kPa.

Not yet used: the NIST 1.7 K cooler (Kotsubo) gives $P_U = 0.2$ MPa, $P_L = 3.2$ kPa, 0.6 mg/s, precoolers at 80, 25 and 10 K, 1.4 mW at 1.7 K, but no CHEX effectiveness; and the He-3 cryocooler of Shimazaki et al. (1 mW at 0.65 K, 160 μmol/s, about 0.48 mg/s, hence $q_C \approx 2.1$ kJ/kg) gives neither P_U nor effectiveness. They allow a rough consistency check only.

What the validation does and does not show

The model reproduces the paper's He-3 results and the paper-level idealised numbers of a second, independent He-4 cooler, so the energy-balance logic ($Q_{\max}$, effectiveness, $q_C = \min(\Delta h_T, q_L)$) and the property tables are consistent with published work. It does *not* predict measured performance to better than the heat leak and pressure drop it omits, as the Ma cases show (differences from about −10 % to +200 %, depending on how close the operating point is to the critical point or to the no-load limit). Adding a return-line pressure drop and a heat-leak term would be the natural next step if measured performance is to be predicted.